Tham You Jie - Project Portfolio Page

Overview

Business Management System (BMS) is a Java command-line application for small retail business owners who handle day-to-day operations themselves. Designed for users who type quickly and prefer typed commands, it brings product, stock, order, payment and expense tracking together in one application. My contributions focus on helping these owners inspect customer purchases, record payments and find the commands they need, together with documenting these features for future developers.

Summary of Contributions

Code contributed: tP Code Dashboard

Feature: Listing sales (sales list)

What it does: displays non-cancelled orders and their combined sales value.

Justification: gives the owner a sales summary without manually adding individual order amounts.

Highlights: uses the shared order history so cancellations are reflected in subsequent listings. Fixed the display numbering to remain consecutive after cancellation while preserving the original order IDs, and updated the regression test. Later reused the shared long-valued unit total to prevent sales quantities overflowing into negative numbers (#27, #67).

Feature: Generating receipts (generate receipts c/CUSTOMER)

What it does: displays individual receipts for a customer's non-cancelled orders.

Justification: lets the owner retrieve purchase details when serving a customer without searching the complete order list.

Highlights: separates customer lookup and receipt selection from UI formatting, supports full names containing spaces and case-insensitive matching, and distinguishes an unknown customer from one whose orders are all cancelled. Reuses the order table layout and returns an unmodifiable result list (#26).

Feature: Recording customer payments (order add pay c/CUSTOMER a/AMOUNT)

What it does: introduced customer payment recording, change calculation and paid-order status.

Justification: separates recording an order from recording the customer's payment.

Highlights: implemented a dedicated parser requiring the customer before a valid monetary amount, with decimal arithmetic for payment and change (#64). After a teammate added partial payments, fixed discounted outstanding balances, settlement of multiple free orders and payment-log accuracy, with regression tests (#102). Partial-payment support itself was the teammate's contribution.

Feature: Viewing customer order details (order details c/CUSTOMER)

What it does: displays each matching order with its products, quantities, costs and status.

Highlights: retains cancelled orders for reference, keeps each order separately identified, and reports when no orders match. Separates input validation, order selection and table presentation (#65).

Feature: Command help (help [AREA])

What it does: displays command descriptions and syntax for a selected area.

Highlights: supports case-insensitive areas, an overview when no area is given, helpful errors for unknown areas and square brackets for optional arguments. Keeps help content in HelpCommand and formatting in Ui (#66).

Testing and diagnostics: added JUnit coverage for sales filtering and totals, receipt lookup and result-list behaviour, and the payment, order-details and help commands. Added Java assertions and logging to sales and receipts to check internal assumptions and report execution outcomes (#39, #40).

Documentation:

User Guide: contributed the sections for sales listing, receipt generation, customer payments, individual order details and help, including syntax, validation rules and examples.

Developer Guide: contributed the Sales and customer commands sections and command-help documentation, covering responsibilities, execution flows, alternatives and integration limitations. Initially added five sequence diagrams (#75); later replaced the order-details sequence diagram with a class diagram and refined UML notation following review feedback.

Contributions to team-based tasks:

Helped resolve UI output/test indentation mismatches and command-signature integration problems during merges (#23, #26).

Contributed test-coverage improvements (#72) and updated the About Us page (#29).

Review/mentoring contributions:

Requested an excess-payment regression test in the financial-reporting PR (review).

Checked whether renaming the exit command was reflected in help text (review).

Reviewed assertion readability in expense tests (review).

Contributions beyond the project team:

Reported a data-loss bug in another team's JobSeek application: ending input during a notes edit erased existing notes and persisted the deletion. Provided reproduction steps, expected and actual behaviour, impact and a suggested fix (bug report).